

ONBOARDING REPORT

Reddit Clone

A grounded, file-by-file walkthrough of the backend (Spring Boot modular monolith) and frontend (React / Vite) — written for an engineer whose prior experience is basic Spring Boot CRUD (@RestController/@Service/@Repository/JPA) and nothing beyond it.

Every claim in this report is grounded in the real code in this repository — exact file paths and line numbers are cited throughout. Where a design choice here differs from the simplest possible approach, that difference is called out explicitly: what the naive option would have been, why it wasn't used, and what problem the real approach solves.

Contents

1. Tech Stack

Spring Boot · Spring Security · PostgreSQL · Flyway · Redis · WebSocket/STOMP · Argon2 · Docker Compose · React · Vite · TypeScript · react-router

2. Architecture

The modular monolith & ArchUnit · Request flow: submitting a post · The Outbox pattern & scheduled jobs · The media upload pipeline · Auth: Argon2, JWT, CORS

3. Key Design Decisions

A running log: the naive option, what we built instead, and why — including two real bugs found this phase

4. Feature Report

Every shipped feature in build order · what's next · what's parked · what's permanently out of scope

5. Glossary for Self-Study

Every non-CRUD concept used in this codebase, with a pointer to where to see it in real use

1. Tech Stack

For each technology: what it is (assuming only basic Spring Boot CRUD background), why *this project* specifically needed it, and exactly where it's configured.

Spring Boot

What it is: the application framework that assembles a runnable Java web app — an embedded Tomcat server, dependency injection, and auto-configuration driven by "starter" dependencies, so you declare *what* you need (web, JPA, security...) instead of hand-wiring it.

Why this project needed it: it's the chassis every other piece below plugs into. Worth flagging: this project runs **Spring Boot 4.1.1 on Java 25** — genuinely current, not a safe "LTS minus one" choice — which has already had real consequences: Boot 4 split Flyway's autoconfiguration and Jackson out of the base web starter into their own separate starter dependencies, both of which had to be added explicitly after the app silently failed to run migrations / failed to compile. A useful lesson: being on the newest framework version means occasionally re-deriving things tutorials still assume are automatic.

- Entry point: `RedditCloneApplication.java:6-11` — `@SpringBootApplication + SpringApplication.run(...)`.
- Version: `pom.xml:5-9` (spring-boot-starter-parent 4.1.1), `pom.xml:30` (java.version 25).
- Starters: `pom.xml:47-79` — actuator, data-jpa, security, validation, webmvc, websocket, and an explicitly-added spring-boot-starter-jackson.
- Port override: `application.yml:1-2` — 8081 (8080 was already in use on the dev machine).

Spring Security

What it is: a chain of servlet filters sitting in front of every controller, deciding "who is this" (authentication) and "are they allowed to do this" (authorization), configured declaratively via a `SecurityFilterChain` bean rather than hand-written checks inside each controller method.

Why this project needed it: this is a stateless, JWT-bearer-token REST API consumed by a separate-origin SPA — almost none of Spring Security's session-cookie-app defaults apply as-is, and nearly every one had to be deliberately reconfigured (full walkthrough in Section 2): CORS opened for the Vite dev origin, CSRF disabled, sessions set stateless, a custom `JwtAuthFilter` added, and the default "403 for unauthenticated" behavior overridden to 401.

- Full configuration: `SecurityConfig.java` — CORS (lines 35–50), CSRF disabled (line 55), stateless sessions (line 57), custom 401 entry point (lines 58–62), error-dispatch exemption (lines 64–69), public vs. authenticated routes (lines 70–85), filter wiring (line 86).

PostgreSQL

What it is: the relational database storing every piece of durable data.

Why this project needed it: a genuinely relational domain, plus three Postgres **extensions** this project leans on directly: `citext` for case-insensitive unique columns (so `AskReddit` and `askreddit` are the same name), `pg_trgm` for fuzzy community-name search, and `ltree` for hierarchical comment paths. The `citext` choice produced a real bug: `pgjdbc`, by default, binds a Java `String` as a concrete `varchar` parameter, which Postgres then has to cast the column down to plain text to compare — silently making username/name lookups case-*sensitive* even though the column itself is case-insensitive at rest. Fixed with `?stringtype=unspecified` on the JDBC URL (commit `daec834`).

- Extensions: `V1__users.sql:1-3`.
- App datasource (through `PgBouncer`, with the fix): `application.yml:7-17`.
- Flyway's own datasource (bypasses `PgBouncer`): `application.yml:18-21`.
- Docker service: `docker-compose.yml:1-14` — `postgres:18`, host port 5434 (5432 was taken locally).

Worth knowing: the app never talks to Postgres directly for normal traffic — every request goes through **PgBouncer**, a connection pooler (`docker-compose.yml:18-28`), while Flyway migrations connect straight to Postgres, bypassing the pooler, since Flyway needs an advisory lock Hibernate doesn't.

Flyway

What it is: a schema-migration tool — instead of Hibernate auto-generating/altering tables (`ddl-auto: update`, dangerous and non-reproducible across environments), you write numbered, immutable SQL files applied once each, in order.

Why this project needed it: the schema evolved across four build phases over **20 migrations**, several of which exist specifically to *fix* an earlier migration's bug once it was actually exercised live (e.g. a table declared `PARTITION BY RANGE` since Phase 1 with zero partitions ever created — fixed only once something finally wrote to it). `spring.jpa.hibernate.ddl-auto` is explicitly `validate`, not `update` — Flyway owns the schema, Hibernate only checks its mappings match.

- Migrations: `src/main/resources/db/migration/V1-V20`.
- Hibernate deferring to Flyway: `application.yml:22-24`.
- Dependency: `pom.xml:31,91-102`.

Redis

What it is: an in-memory key-value store, used here as a cache — accessed via Spring Data Redis's `StringRedisTemplate` directly (hand-rolled, not the `@Cacheable` annotation abstraction).

Why this project needed it: a two-level cache in front of the single most expensive, most-hit read in a Reddit-like app — page 1 of /hot, anonymous, no cursor — which includes **r/all**, the sitewide aggregate-across-every-community feed built during the frontend phase. An in-process Caffeine cache (5s TTL, fast

but per-instance) is backed by Redis (45s TTL, shared) and the whole thing **fails open** — a Redis outage falls back to a plain DB read instead of a 500, since the feed worked before this cache existed.

- Dependency: `pom.xml:87–90`. Connection: `application.yml:26–29` — `localhost:6380`.
- Docker service: `docker-compose.yml:15–17` — `redis:8`, host port 6380 (6379 was taken locally).
- Cache implementation: `FeedCacheService.java` — design rationale (lines 14–17), Caffeine layer (24–25), Redis read/write with fail-open (34–60).
- Wired in: `PostController.java:89–127` — consulted only for page 1 + an unauthenticated viewer.

WebSocket / STOMP

What it is: WebSocket upgrades an HTTP connection into a persistent, two-way connection — the server can push data at any time, no polling. STOMP is a lightweight messaging sub-protocol on top (SUBSCRIBE, SEND, named destinations), giving `@MessageMapping` methods a role analogous to `@RequestMapping` for REST.

Why this project needed it: the real-time chat feature (which replaced classic store-and-forward Private Messages — see Section 4) needs to push a new message to the recipient instantly. The genuinely tricky part: a browser's native WebSocket handshake is a plain HTTP GET that JavaScript **cannot** attach a custom `Authorization` header to. The fix: let the HTTP handshake through unauthenticated, then authenticate one layer up, at the first STOMP CONNECT frame (which *can* carry custom headers), via a `ChannelInterceptor` that fails the connection outright if the token is missing or invalid.

- Dependency: `pom.xml:66–71`.
- Config + the CONNECT-frame auth interceptor: `ChatWebSocketConfig.java` — problem explained (18–25), endpoint registration (36–41), broker config (43–51), interceptor (53–79).
- Public HTTP route for the handshake: `SecurityConfig.java:81–84`.

Argon2 (password hashing)

What it is: a deliberately slow, memory-hard password-hashing algorithm — the opposite design goal of a fast hash like SHA-256, which an attacker can test billions of times per second against a stolen hash table on a GPU.

Why this project needed it: any app storing passwords needs resistance to offline brute-force if the database is ever compromised. Full walkthrough in Section 2.

- Bean: `SecurityConfig.java:27–30` — new `Argon2PasswordEncoder(16, 32, 1, 19456, 2)`, tuned for ~100–200ms per hash.
- Supporting dependency (BouncyCastle, not pulled in automatically): `pom.xml:155–162`.
- Call sites: `AuthService.java` — registration (59), login verification (72), account-deletion re-verification (238, 243).

Docker Compose

What it is: a single YAML file defining a multi-container local dev environment, started together with one command.

Why this project needed it: four separate infrastructure pieces locally (database, cache, connection pooler, object storage). Several ports are deliberately non-default, each with an inline reason:

- **postgres** — `postgres:18`, host port 5434 (5432 taken locally), healthcheck.
- **redis** — `redis:8`, host port 6380 (6379 taken locally).
- **pgbouncer** — session pooling; needed `LISTEN_PORT: 6432` (the image defaults its internal listen port to 5432 regardless of the published port) and `AUTH_TYPE: scram-sha-256` (Postgres 18 defaults to SCRAM; the image's own default of md5 fails auth against it).
- **s3mock** — `adobe/s3mock`, an S3-compatible mock store for the media pipeline; chosen over the originally-planned MinIO because MinIO's Docker Hub image started requiring auth to pull, even for old pinned tags.

Full file: `docker-compose.yml`.

React

What it is: a component-based UI library — the interface is a tree of reusable components that re-render automatically when their state changes.

Why this project needed it: the frontend is a full SPA, decoupled from the backend, talking to it purely over REST + WebSocket — React is the standard choice for this kind of client-rendered, highly-interactive UI (voting, live chat, nested comment trees).

- Dependencies: `frontend/package.json:12-16` — `react ^19.2.8`, `react-dom ^19.2.8`, `react-router-dom ^7.18.4`.
- Entry point: `frontend/src/main.tsx`.

Vite

What it is: a frontend build tool and dev server — near-instant hot-reload in development, an optimized bundle for production.

Why this project needed it: it's the dev server/bundler for the React frontend. One project-specific detail: **the dev server is pinned to port 5174, not Vite's default 5173**, with `strictPort: true` — because the backend's CORS allowlist hardcodes `http://localhost:5174`. Change this port in one place without the other, and the frontend starts fine while every API call silently fails CORS.

- Config: `frontend/vite.config.ts:6-9`. Matching backend origin: `application.yml:54`.

TypeScript

What it is: a superset of JavaScript adding static types, catching mismatches at compile time instead of at runtime in the browser.

Why this project needed it: for an SPA calling a typed REST API, TypeScript catches an entire class of bugs before the code ever runs. The build script (`tsc -b && vite build`) runs the compiler as a build gate *before* bundling.

- Config: `frontend/tsconfig.json`, `tsconfig.app.json`. Build script: `frontend/package.json:8`.

react-router (react-router-dom)

What it is: client-side routing for SPAs — intercepts URL changes and swaps which component renders, without a full page reload. Plays the role client-side that `@GetMapping("/path")` plays server-side.

Why this project needed it: the frontend is a genuine multi-page experience (home feed, login, a community page, post-detail-and-comments, post submission, community discovery/creation) served from one SPA bundle.

- Router setup: `frontend/src/main.tsx` — `<BrowserRouter>` wraps `<AuthProvider><App /></AuthProvider>`.
- Route table: `frontend/src/App.tsx` — including two different routes (`/submit` and `/r/:communityName/submit`) that both render `PostSubmit`, branching internally on whether a community was pre-selected.

Other dependencies worth knowing about

- **ArchUnit** — mechanically enforces the modular-monolith boundaries described in Section 2; not just a documented convention.
- **UUIDv7** — every generated ID is a time-ordered UUID (sorts naturally, friendlier to index locality) rather than random UUIDv4 or auto-increment.
- **ShedLock** — distributed locking for `@Scheduled` jobs, so a job never runs concurrently from two app instances.
- **Bucket4j** — a rate-limiting library already in `pom.xml` but not yet wired to any endpoint (see Section 4).

2. Architecture

Written assuming only basic CRUD experience. Every claim cites an exact file and line — read the real code alongside this explanation.

The modular monolith: package structure and ArchUnit

The whole app is **one deployable** — one JVM, one `.jar`. That's the "monolith" half. The "modular" half is a convention this codebase enforces much more strictly than most: code is split into package-per-feature modules, and a module may only call another module's `@Service` class — never reach directly into its `@Repository`. This gets most of the separation benefit people reach for microservices for, without any network calls or deployment complexity.

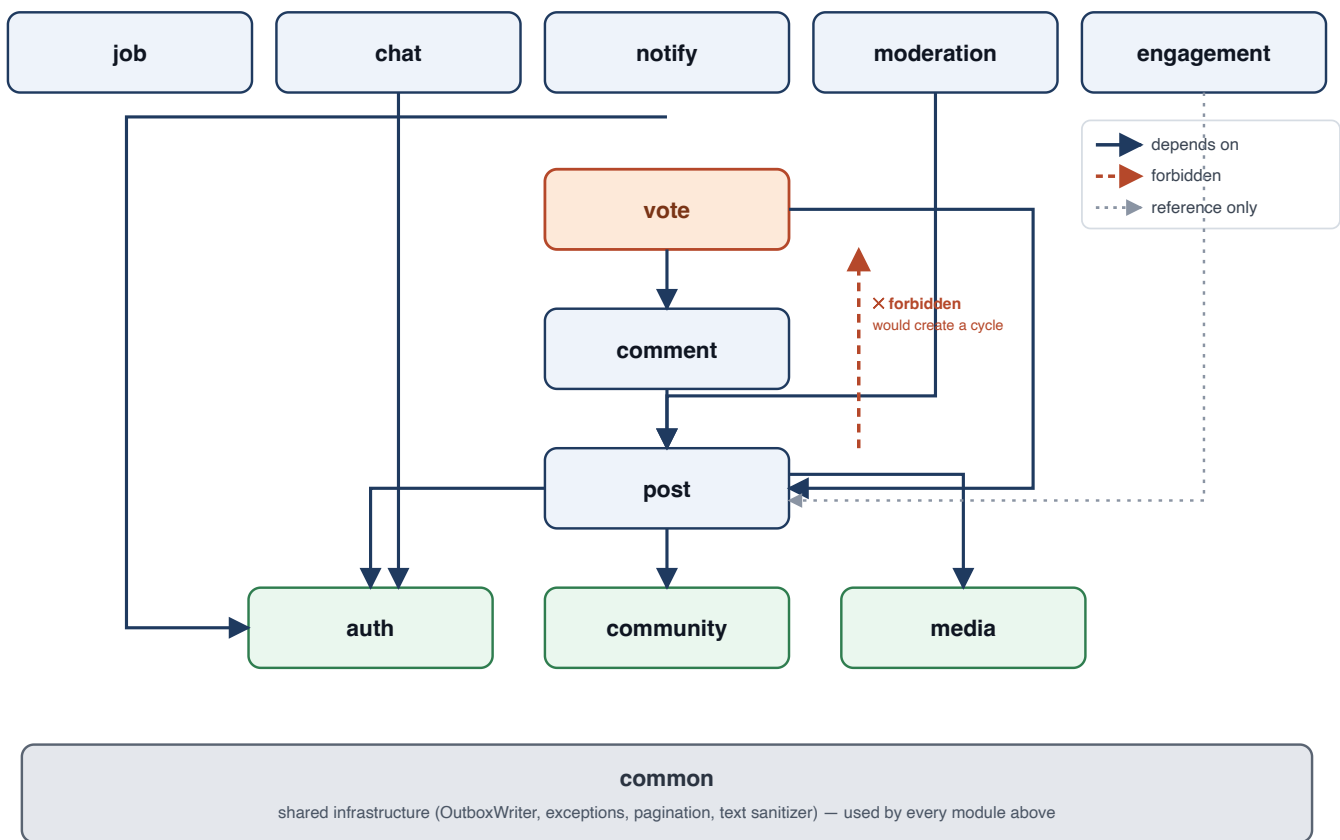


Fig. 1 — Module layout. Arrows not drawn for space: `comment` → `community`, `comment` → `auth`, `moderation` → `community`, `moderation` → `comment` (same pattern as the ones shown). `job` has no service-level dependencies — it runs scheduled raw-SQL statements directly against tables.

Package	Owns
auth	User identity, credentials (Argon2), JWT issuing/parsing, refresh-token rotation, site-wide admin actions
post	The <code>Post</code> entity, submission, all feed listings/sorts, search, pin/lock, the <code>/hot</code> cache
comment	The <code>Comment</code> entity, nested reply tree, comment sort options
community	Communities, membership, moderators, bans, flairs, automod rules, join requests
vote	Upvote/downvote state, the vote outbox worker, karma ledger — kept deliberately isolated
moderation	Human moderator actions: reports, mod queue, mod-mail, the moderation action log
notify	User-facing notifications and their own outbox-drain worker
engagement	Saved/hidden items — small, generic, cross-cutting per-user state
media	Uploads, presigned URLs, image/video processing workers
chat	Real-time direct messaging over STOMP/WebSocket
job	Scheduled jobs not tied to one module's own write path (rank decay, nightly reconciliation)
common	Genuinely shared infrastructure: the outbox writer, exceptions, pagination types, text sanitization

The ArchUnit rule that enforces this mechanically

File: `ModuleBoundaryTest.java`. ArchUnit is a library that lets you write architecture rules as ordinary JUnit tests — it statically scans your *compiled classes* and checks structural rules like "no cycles between these packages." A violation fails the build the same way a broken unit test would.

Two kinds of rules are defined. First, cycle-freedom across every top-level module:

```
@ArchTest
static final ArchRule modules_are_free_of_cycles =
    SlicesRuleDefinition.slices().matching("com.redditclone.
    (*).").should().beFreeOfCycles();
```

Second, ten near-identical rules — one per module:

```
private static ArchRule repositoryAccessRule(String modulePackage) {
    return classes()
        .that().resideInAPackage(modulePackage)
        .and().haveSimpleNameEndingWith("Repository")
        .should().onlyBeAccessed().byClassesThat()
            .resideInAnyPackage(modulePackage, "..common..");
}
```

In plain English: any Spring Data repository interface in module X can only be injected into classes in module X (or common). No module may autowire another module's repository directly — it must go through

that module's service class.

A concrete example — why vote state isn't a field on Post. The repository rule alone doesn't literally forbid `post` depending on `vote`. What actually makes that impossible is the combination of two facts: `vote.VoteService` already imports `post.PostService` and `comment.CommentService` (to validate a target exists before accepting a vote on it), and the cycle-freedom rule means nothing can depend back on `vote` without creating a forbidden cycle (confirmed — a `grep` across `post/comment` for `import com.redditclone.vote` finds nothing). The visible consequence: a feed or comment-tree response can never carry "did I already vote on this," because `PostService/CommentService` have no legal way to call into `vote`. That's exactly why the frontend makes a **second, separate call** — `GET /api/vote/mine` (`VoteController.java:40-48`) — and composes the two results client-side. The comment directly above that endpoint says so: *"Lets the frontend render correct vote-arrow state after a reload — the feed/comment-tree endpoints themselves can't carry this."*

This test once caught a real mistake during the original build: `SecurityConfig` was initially placed in the shared common package, creating a cycle (`common` depending on `auth.JwtAuthFilter`, while `auth` depended on `common` for exceptions) — moved into `auth` to fix it, since it's fundamentally `auth`'s own configuration.

Request flow: submitting a post, end to end

Walking `POST /r/{communityName}/submit` through every layer.

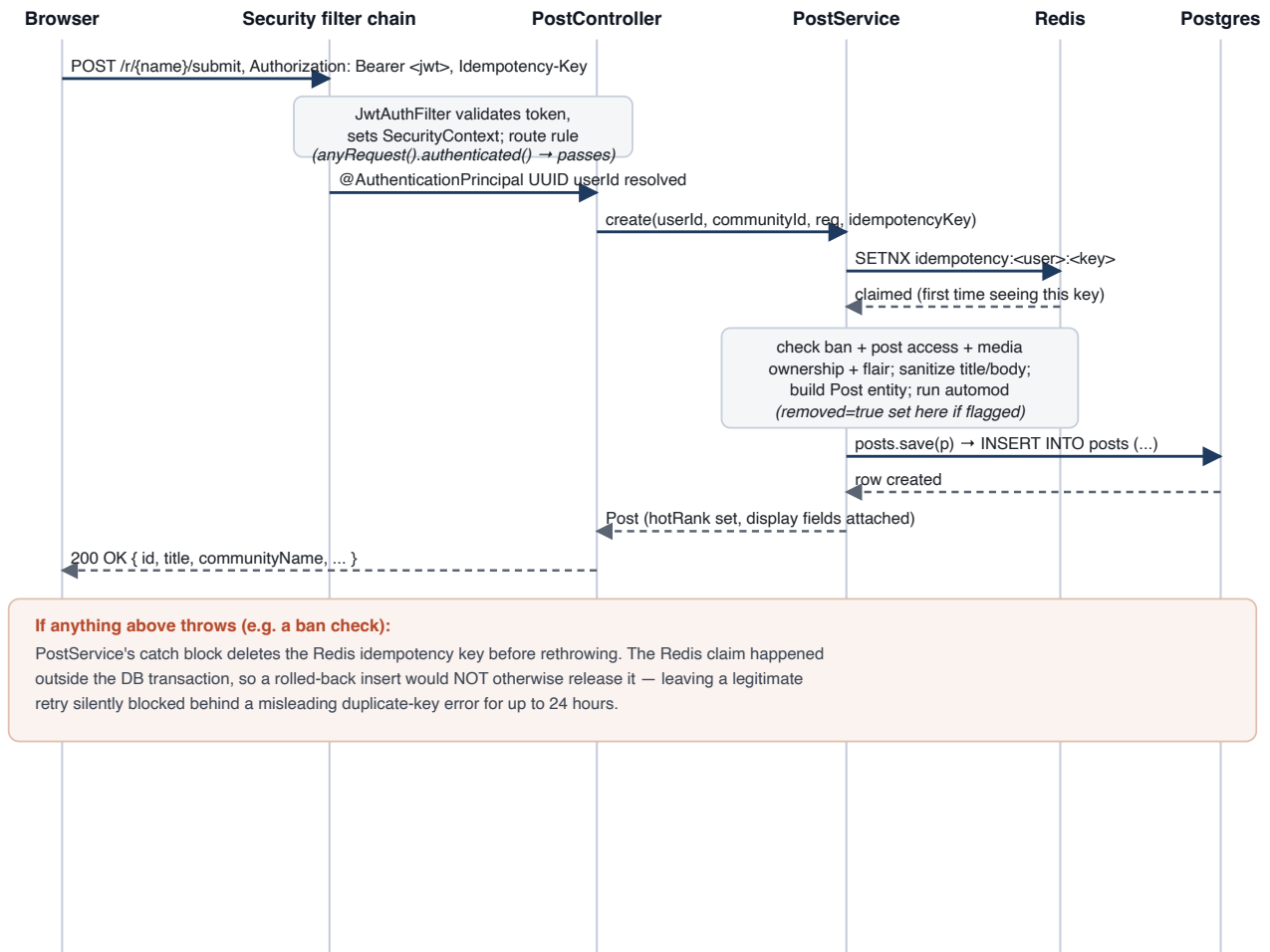


Fig. 2 — `POST /r/{communityName}/submit`, end to end. Solid arrows = calls; dashed arrows = returns.

DTOs vs. entities — why there are two classes for "a post"

`CreatePostRequest.java` is a Java record with exactly 8 fields — `kind`, `title`, `body`, `url`, `mediaId`, `flairId`, `nsfw`, `spoiler` — plus Bean Validation annotations and a few cross-field checks (e.g. a link post requires a `url`). `Post.java` is the full JPA entity with roughly 20 fields, most of which the client never sends: `score`, `commentCount`, `hotRank`, `risingRank`, `pinned`, `locked`, `removed`, plus `@Transient` display-only fields populated only at read time.

Why not just use `Post` as the request body directly?

1. **Mass-assignment safety.** If a client could POST a raw `Post` JSON body, nothing would stop it setting `score=999999` or `removed=false` on creation. The DTO has no field for those — there's nothing to bind.
2. **Validation belongs on the input shape**, not the persisted shape — "a link post needs a URL" is a rule about what a client may submit, not an invariant of every stored row (a post rehydrated for display never goes through these checks).

3. **The entity carries JPA concerns** — setters deliberately restricted to one legitimate caller (score has no public setter at all; it's only mutated via the batched vote-processing update described below) — that have no business being exposed over the wire.

`PostService.create` is the translation layer: it validates the DTO, then explicitly copies individual fields onto a freshly constructed entity — never accepting or returning an entity as a raw, client-controlled request body.

Async and event-driven pieces: the Outbox pattern and scheduled jobs

In basic CRUD, a controller method does its one job and returns. Here, several user actions need to trigger slower downstream work without the request waiting on it. Two mechanisms handle this: the **Outbox pattern** for work triggered by a specific user action, and **@Scheduled jobs** for correcting things on a clock, independent of any request.

The Outbox pattern, via vote processing

When a user upvotes a post, two things need to happen: (1) record the vote, (2) update score/karma/rank. The outbox pattern splits these — step 1 happens synchronously and also writes a small "this happened" event row **in the same database transaction**, so neither can be lost without the other; step 2 happens later, batched, in a background worker.

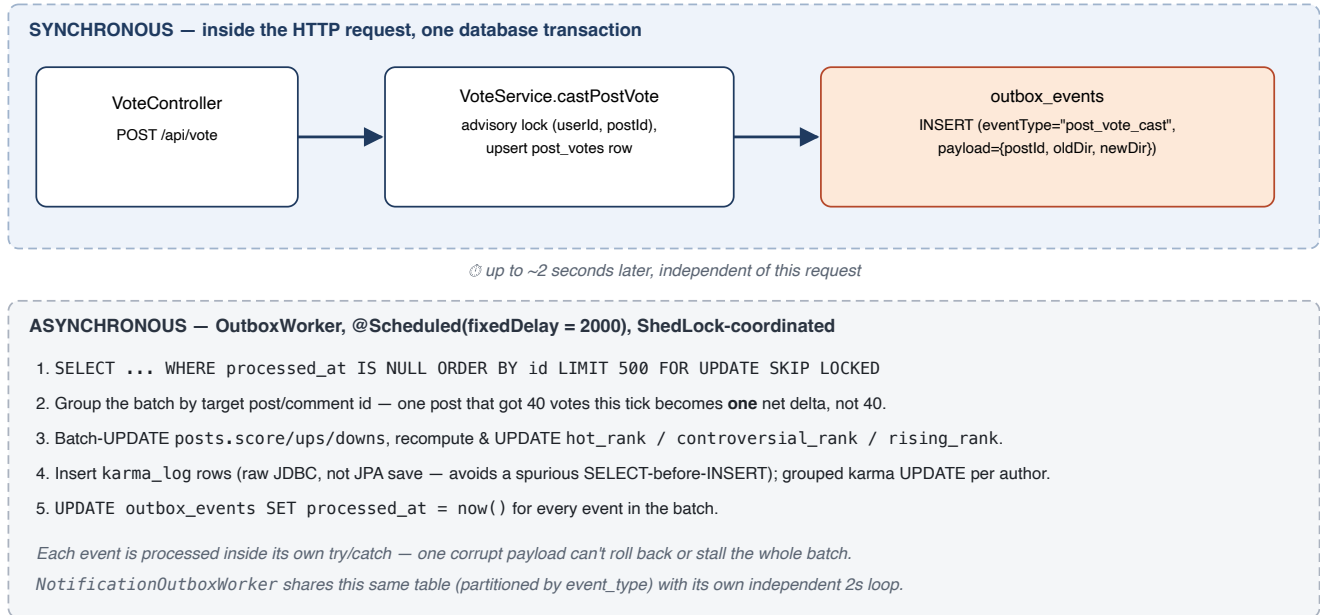


Fig. 3 — The Outbox pattern: cast a vote now, apply its downstream effects in the next batch.

Why not synchronous? (1) Unpredictable fan-out cost — a vote can trigger an UPDATE, a re-read, three rank recomputations, a karma insert, and an author UPDATE; inline, every click would wait on all of it. (2) Batching only works when deferred — grouping 40 votes into one UPDATE is only possible because they

accumulate first. (3) Partial-failure isolation — a bad event can't roll back or delay the original request, which already committed independently. (4) Room to scale out later — the `FOR UPDATE SKIP LOCKED` design means this can later run across multiple app instances with no rewrite. The cost: a small window ($\leq \sim 2s$) where a fresh vote hasn't yet moved the numbers — an accepted trade-off, not an oversight.

Scheduled jobs: correcting things that aren't tied to any one request

All share the same shape — `@Scheduled` (when), `@SchedulerLock` (only one instance runs it), wired up by `SchedulingConfig.java`, which also points the lock provider at the **database's own clock** to avoid clock-skew bugs across instances.

- **RankDecayJob** (`RankDecayJob.java`) — every 5 minutes, halves every post's `rising_rank` (a *velocity* measure — "votes per minute, right now"). The instant voting slows down, that number goes stale; there's no future request that would naturally recompute it, since viewing a post doesn't change its vote velocity. Something has to walk the table on a clock.
- **ReconciliationJob** (`ReconciliationJob.java`) — nightly at 03:30. This project deliberately omits foreign keys on `post_votes`, `comment_votes`, and `comments`; without FKs, nothing stops or flags an orphaned row. This job sweeps for exactly that drift — auto-deleting orphaned votes (safe, no content of their own) but only counting+logging orphaned comments (real content, worth a human look).
- **MediaReaperJob** — covered below, since it's part of the media pipeline's failure-recovery story.

The one idea that unlocks all of these: a `@Scheduled` job exists whenever a problem is *not request-shaped* — there's no single incoming request whose handler could reasonably notice and fix it, either because the trigger is time itself (decay), the database won't catch the drift on its own (reconciliation), or the process that was supposed to finish the work might not exist anymore (a crash mid-upload).

The media upload pipeline

Every basic file-upload tutorial looks like `@PostMapping(consumes = MULTIPART_FORM_DATA_VALUE) void upload(@RequestParam MultipartFile file)`. This project does **not** do that anywhere. The raw bytes never pass through this Spring backend — the backend's only job is to (a) hand out a short-lived signed URL pointing at the object storage service, and (b) track state in Postgres.

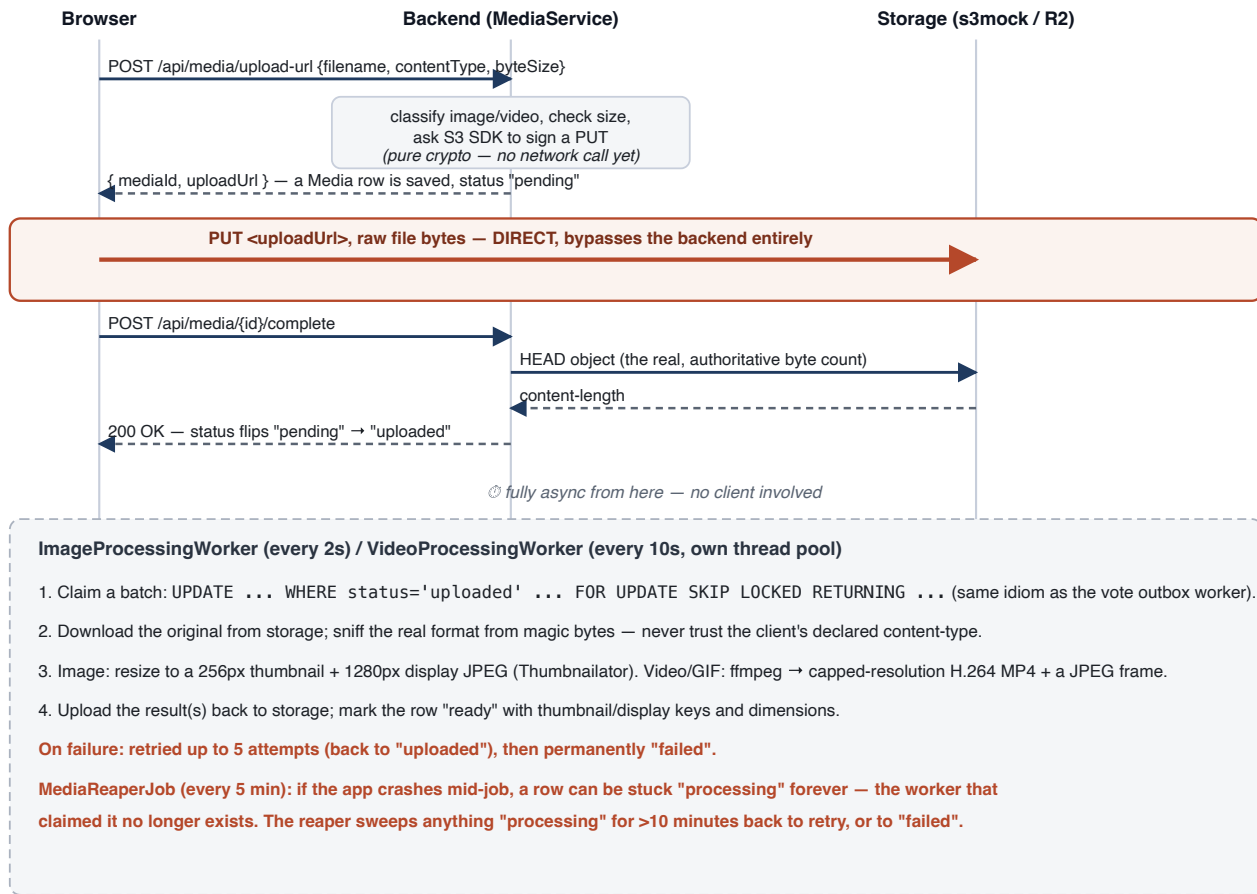


Fig. 4 — The presigned-upload flow. The 20MB/200MB file transfer never touches a Tomcat thread, the app's heap, or its network link.

Why presigned URLs instead of MultipartFile? (1) Byte ceilings here are real and large (20MB images, 200MB videos per `application.yml:47-48`) — routing that through a multipart endpoint means the whole file flows through a Tomcat request thread and the JVM heap before the app can do anything with it. (2) The app's job shrinks to "sign a request," not "move bytes" — presigning is pure in-memory cryptography that never even contacts storage, versus proxying, which would mean receiving the whole file and then making its own outbound call to push it on — doubling the transfer. (3) Verification happens where it's cheap and authoritative — one HEAD request at completion, not streaming and counting every byte in the app itself.

Auth: Argon2, JWT access + refresh rotation, and CORS

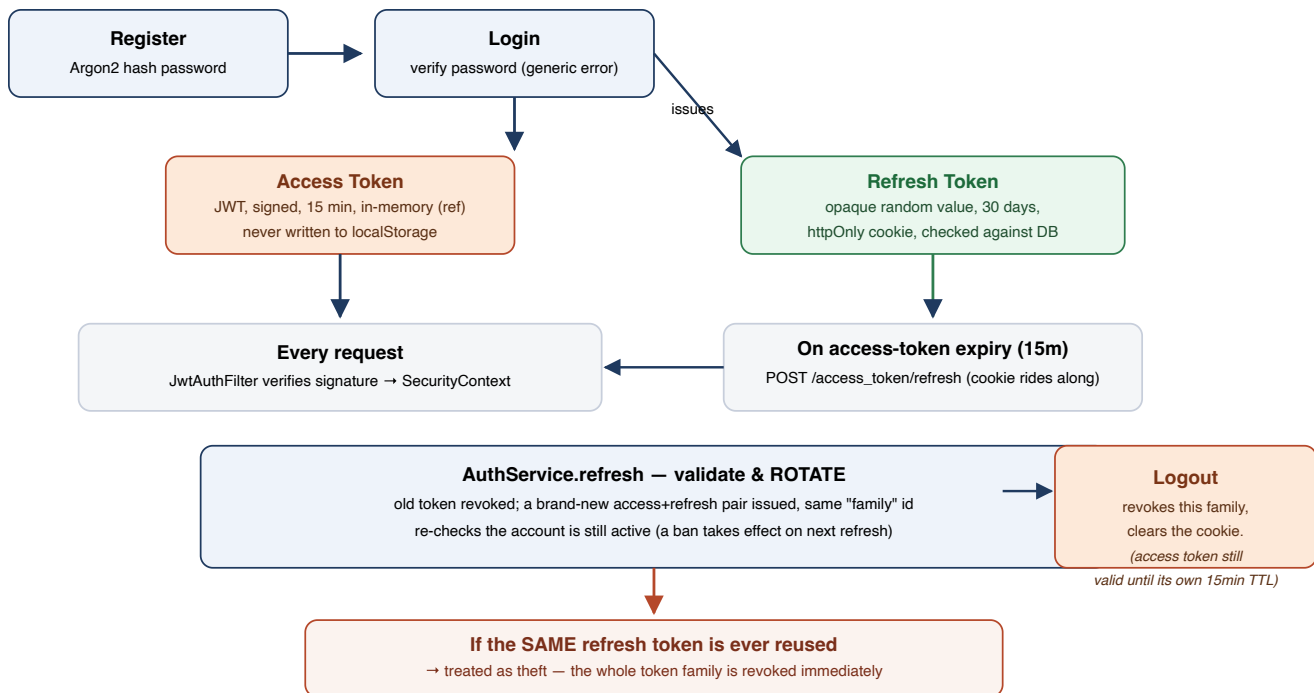


Fig. 5 — Token lifecycle: two tokens with two different jobs, rotated and revocable independently of each other.

Registration — Argon2 password hashing

```
// SecurityConfig.java:27-30
@Bean
public PasswordEncoder passwordEncoder() {
    return new Argon2PasswordEncoder(16, 32, 1, 19456, 2); // tuned for ~100-200ms
}
```

Deliberately slow and memory-hungry (16-byte salt, 32-byte hash, parallelism 1, ~19MB memory, 2 iterations) — imperceptible for one real login, but makes brute-forcing a stolen hash table prohibitively slow, unlike a fast hash like SHA-256. Call site: `AuthService.java:59` — `user.setPasswordHash(encoder.encode(rawPassword))`.

Login — two tokens, two jobs

`AuthService.login (AuthService.java:69)` returns the exact same generic "invalid credentials" message whether the username doesn't exist *or* the password is wrong, and checks the account's active/banned status only **after** verifying the password — both deliberate, to avoid leaking account information to a guesser without the right password.

	Access token	Refresh token
What it is	Signed JWT	Opaque random value — not a JWT

Lifetime	15 minutes	30 days
Verified how	Self-contained signature check, no DB lookup	SHA-256 hash looked up against a DB row on every use
Where it lives	In-memory in the frontend, never persisted	An httpOnly cookie
Why	Fast to verify, short-lived so theft is briefly useful at worst	Long-lived and powerful, so never exposed to JavaScript

A JWT's only claims are sub (user UUID) and standard iat/exp — no roles embedded. Authorization elsewhere (e.g. "is this a site admin?") re-queries the database, never trusts claims baked into the token.

Every authenticated request — JwtAuthFilter

Spring Security works as a **filter chain** — before a request reaches a controller, it passes through filters assembled in `SecurityConfig.filterChain (SecurityConfig.java:52)`. This project's own filter is spliced in explicitly: `.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class) (SecurityConfig.java:86)`. On every request, `JwtAuthFilter.java` checks for `Authorization: Bearer <token>`; if valid, it installs the user's UUID into Spring Security's `SecurityContext` — a per-request holder any later code reads via `@AuthenticationPrincipal UUID userId`. If the token is missing/expired/tampered, the filter does nothing — rejection, if any, happens downstream when the route's authorization rule is checked. The whole mechanism is **stateless** (`SecurityConfig.java:57`) — nothing about "who's logged in" is remembered between requests; every request re-proves its identity from scratch.

Why 401, not Spring Security's default 403

```
// SecurityConfig.java:58-62
// Spring Security's default anonymous-authentication + access-denied handling returns 403
// for a
// missing/invalid token on an authenticated-only route; a bearer-token API should return 401
// there
// instead (403 is reserved for an authenticated principal lacking permission).
.exceptionHandling(ex -> ex.authenticationEntryPoint(new
HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
```

Out of the box, Spring Security treats an unauthenticated caller as a harmless "anonymous" principal, so a denied route surfaces through the *access-denied* path, which defaults to 403. This project overrides that so a missing/invalid token on a protected route returns 401 instead — reserving 403 for "you're logged in, but not allowed to do this specific thing."

CORS: allowedOrigins vs. allowedHeaders are two separate dials

```
// SecurityConfig.java:35-50
config.setAllowedOrigins(List.of(allowedOrigins.split(",")));
config.setAllowedHeaders(List.of("Authorization", "Content-Type", "Idempotency-Key"));
config.setAllowCredentials(true);
```

Because the frontend (`localhost:5174`) and backend (`localhost:8081`) run on different origins, the browser sends an automatic `OPTIONS` preflight asking permission first. `allowedOrigins` controls which *sites* may call the API; `allowedHeaders` separately controls which *request headers* the real request may carry. These are independent — and this project hit a real bug because of it (see Section 3). `allowCredentials(true)` is also why `allowedOrigins` can never be a wildcard `"*"` — browsers reject that combination outright.

3. Key Design Decisions

A running log of places this project did something other than the simplest possible thing, in the order they came up. For each: what the naive approach would have been, what was actually done, and why — because the "why" is usually a real bug or a real constraint, not a stylistic preference.

1. Java records as request DTOs — and the null-vs-primitive gotcha they create

SIMPLE OPTION: use `boolean` for a flag like "is this NSFW," the same as on any JPA entity.

WHAT HAPPENED: `CreatePostRequest.java` is a record. When Jackson deserializes JSON into a record, it calls the **canonical constructor** directly — there's no JavaBean-style "skip the setter if absent" step. An omitted JSON field becomes `null` passed into the constructor. Adding `boolean nsfw`, `boolean spoiler` as primitives broke **almost every existing post-creation call in the whole regression suite** the moment it shipped — every pre-existing script's JSON predates these fields, and Jackson crashed trying to unbox `null` into a primitive.

THE FIX — `Boolean` (boxed) fields plus a compact canonical constructor normalizing `null` to a default, right at construction:

```
public record CreatePostRequest(..., Boolean nsfw, Boolean spoiler) {  
    public CreatePostRequest {  
        if (nsfw == null) nsfw = false;  
        if (spoiler == null) spoiler = false;  
    }  
}
```

This is exactly why the house style runs the **full** existing regression suite after every change, not just new checks — this was caught immediately because of that discipline. Any future primitive field added to an existing client-deserialized record needs the same boxed-type treatment.

2. Why "my vote state" is a separate endpoint, not a field on Post

SIMPLE OPTION: add a myVote field to Post/Comment, populated on every fetch.

WHAT HAPPENED: vote already depends on post/comment (it must confirm a target exists before accepting a vote). ArchUnit's cycle-freedom rule forbids post/comment depending back — structurally, not just by convention.

THE FIX: a dedicated endpoint living entirely in vote — `GET /api/vote/mine` — that the frontend calls as a second request, composed client-side. See Section 2 for the full ArchUnit walkthrough.

This "entity needs viewer-specific context it doesn't carry" situation recurred twice more during the frontend build (comment reply trees; a community's membership/moderator/join-request status), both solved by attaching the context server-side (batched for lists) rather than a separate endpoint — because those modules don't have a boundary conflict the way vote does. Which pattern applies comes down entirely to whether a module-boundary cycle would result.

3. Redis caching for the hot feed (and r/all specifically)

SIMPLE OPTION: no cache — run the ranked query fresh every request, like a basic CRUD list endpoint.

WHAT HAPPENED: the most expensive, most-hit read in a Reddit-like app is page 1 of /hot, anonymous — which, once a sitewide r/all pseudo-community was built, includes a query aggregating across every community. The project owner explicitly asked for this to be cached the way real Reddit's homepage actually is.

THE FIX: a two-level cache in `FeedCacheService.java` — in-process Caffeine (5s TTL, fast, per-instance) backed by Redis (45s TTL, shared) — scoped to only page 1 for an anonymous viewer (virtually all real traffic). It **accepts staleness** as a deliberate trade-off (a fresh vote doesn't invalidate early) and **fails open** (a Redis outage falls back to a DB read, since the endpoint worked without this cache before).

4. The CORS allowedHeaders gotcha — a 200 preflight doesn't mean the real request will succeed

SIMPLE OPTION: assume a 200 preflight means the real request goes through.

WHAT HAPPENED: submitting a post from the real browser failed with a raw `net::ERR_FAILED` — not even a response reaching application code — despite the `OPTIONS` preflight itself returning 200 and a direct `curl POST` working fine. The request used a custom `Idempotency-Key` header that didn't exist when CORS was first configured. `allowedOrigins` and `allowedHeaders` are two independent allowlists — a 200 preflight status says nothing about whether the specific headers the real request needs are actually listed in that preflight's `Access-Control-Allow-Headers` response value. The browser checks that value and blocks the real request client-side, silently.

THE FIX: add `Idempotency-Key` to `allowedHeaders` (`SecurityConfig.java:45`).

The diagnostic technique is the real lesson: a targeted `curl -i -X OPTIONS` with an explicit `Access-Control-Request-Headers` value reveals the *actual* allowed-headers list — checking only the status code, or testing via `curl POST` (not subject to browser CORS at all), will not catch this.

5. The idempotency-key pattern for post submission

SIMPLE OPTION: insert a new row every time submit is called — a timed-out retry is the client's problem.

WHAT HAPPENED: a timed-out request that actually succeeded server-side, followed by a client retry, would otherwise create a duplicate post. The fix: a client-generated idempotency key (`crypto.randomUUID()`, once per form instance) sent as an `Idempotency-Key` header.

`PostService.create` atomically **claims** that key in Redis (SETNX-style) *before* touching the database; a repeat key returns the already-created post instead of making a second one.

WORTH KNOWING: because the Redis claim lives outside the DB transaction, a failure partway through has to explicitly release the key in a `catch` block — otherwise a rollback leaves the key claimed with no post behind it, blocking a legitimate retry for 24 hours.

6. Client-side validation stricter than the server's, for routing safety rather than fidelity

SIMPLE OPTION: let the frontend accept whatever the backend accepts.

WHAT HAPPENED: a community's name becomes a literal URL route segment (`/r/{name}`) everywhere in the frontend. The backend only enforces length (3–32 chars, any character) — spaces or slashes would silently break routing and links client-side, though the server would never error.

THE FIX: `CreateCommunity.tsx` enforces `^[A-Za-z0-9_]{3,21}$` before ever calling the API, with an inline comment noting this is a routing-safety guard, not a fidelity choice — the backend's own (looser) validation is untouched.

7. Two real correctness bugs in the original outbox-worker build (Phase 2)

WHAT HAPPENED, CAUGHT IN THE SAME PHASE IT SHIPPED: the first version of the vote outbox worker had two real bugs: a vote **switch** (up → down) only ever applied a swing of ± 1 , when it should be a swing of 2; and an **unvote** event was hard-coded to a delta of 0, so removing a vote never reversed its effect on score at all. Both caught and fixed within the same phase.

The fix is encoded in the payload shape itself — `VoteEventPayload.java` deliberately carries *both* the old and new direction, so the worker has enough information to compute a correct net delta for every case.

8. citext and ltree columns need a specific, non-"obvious" Hibernate mapping

SIMPLE OPTION: map an exotic Postgres column type with `@JdbcTypeCode(SqlTypes.OTHER)` on a plain `String` — the mapping that looks most "correct."

WHAT HAPPENED: that mapping passed Hibernate's schema validation but failed on the first real INSERT, because `SqlTypes.OTHER` binds via `setObject(..., Types.OTHER)`, which pgjdbc sends over the wire as untyped binary framing — operator does not exist:
`citext = bytea.`

THE FIX: a plain `String` field, **no** `@JdbcTypeCode` at all, just `@Column(columnDefinition = "citext")` — Hibernate 7 consults `columnDefinition` for validation, and a plain `String` binds via `setString()` (text protocol), which Postgres accepts into a `citext` column via an implicit cast. `ltree` needed one more step (no implicit `varchar`→`ltree` cast exists): `@ColumnTransformer(write = "?::ltree")`. Full investigation: `docs/PHASE_1_BUILD_LOG.md §9.7–9.8`.

9. Spring Security's default is 403 for a missing token, not 401 — and fixing that creates a second bug

WHAT HAPPENED: Spring Security's `AnonymousAuthenticationFilter` treats every unauthenticated caller as a harmless "anonymous" principal, so a denied route is rejected via the *access-denied* path, defaulting to 403. Overriding this with a custom entry point

`(SecurityConfig.java:58–62)` fixes that — but exposed a **second**, self-inflicted bug: an unhandled exception on an unauthenticated request triggers the servlet container's internal forward to `/error`, which **re-enters the same security filter chain as a second, unauthenticated request** — and `.anyRequest().authenticated()` denies *that* one, hiding a real 500 behind a misleading 401.

THE FIX: exempt the `ERROR` dispatch type from the authentication check

`(SecurityConfig.java:64–69)`. Both bugs were found by actually hitting them live (a real `ArithmeticException` surfaced as a misleading 401), not by reading documentation more carefully.

10. A placeholder permission bitmask becomes a real security bug if left unfixed

WHAT HAPPENED: Phase 1 left `CommunityModerator.OWNER_PERMISSIONS = Integer.MAX_VALUE` as a placeholder before a real bitmask system existed. Once Phase 3 built real, named permission bits, granting a regular moderator `Integer.MAX_VALUE` directly (instead of OR-ing together only the named bits) would make them indistinguishable from the literal owner for any "has every `OWNER_PERMISSIONS` bit" check — because any bit anyone adds in the future would automatically evaluate true for them, without anyone deciding to grant it.

THE FIX: grant moderators the OR of only the currently-named constants, never the sentinel value itself.

11. STOMP's `.wrap()` vs. the real inbound accessor — a protocol-layer footgun

WHAT HAPPENED: `StompHeaderAccessor.wrap(message)` builds a **new** accessor object — mutating it does **not** propagate back into the actual message Spring threads through its pipeline. Using `.wrap()` in the chat module's STOMP interceptor silently dropped the authenticated principal on every frame *after* the initial `CONNECT`: a user could connect successfully, but every subsequent chat message arrived unauthenticated. Found only by testing a real send/receive path live — it compiles fine and causes no error, just quietly wrong behavior.

THE FIX: `MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class)`, which returns the real, mutable accessor already attached to the message.

12. Dependency coordinates that look right but don't resolve

WHAT HAPPENED: adding a rate-limiting library by guessing the "obviously latest" coordinates resolved fine in search but **failed at build time** — that artifact only ever published a `.pom` (a dependency aggregator) with no compiled classes behind it. Bucket4j splits its Redis integration by both JDK baseline and client library; the one that actually contains usable classes matches Spring Data Redis's default client.

A dependency that resolves isn't the same guarantee as one that contains the code you need — worth an actual build attempt, not just a version-string guess, with any unfamiliar library.

4. Feature Report

The full build history, in order (see `git log --oneline --reverse`). A pattern recurs across almost every phase: **the database column/table already existed, but zero Java code ever used it** — the schema was designed up front for the whole project, phases ahead of the code that would use each piece. Checking the migrations folder first for an already-existing-but-unused column is a genuinely useful habit this project teaches.

Backend Phase 1 — project skeleton, auth, communities, posts, comments

fa1ff8f, 663601e,
daec834

Problem solved: nothing existed yet — register/login/refresh, create/join/leave a community, submit a post, reply to a comment (depth 10 cap).

Required: the full initial schema (24 tables), the auth/community/post/comment modules, the JWT filter chain, the `ModuleBoundaryTest` ArchUnit rule.

Real bugs: Spring Security's default 403 instead of 401; a follow-up masked-500 bug from fixing that; `citext/ltree` Hibernate mapping gotchas; ArchUnit caught a real circular dependency (SecurityConfig moved from common into auth); a hardening pass found 15 more issues including a duplicate-post race in the idempotency logic. Full detail: [docs/PHASE_1_BUILD_LOG.md](#).

Backend Phase 2 — voting, karma, feed ranking

0fe2334, 59cfd02

Problem solved: posts/comments existed but nothing could be voted on; no /hot, /top, /rising, /controversial.

Required: the vote module (outbox pattern — see Section 2), a job package, precomputed rank columns so feed reads stay plain indexed queries.

Real bugs: two correctness bugs in the first outbox build (vote-switch swing, unvote delta — see Section 3 #7); a code-review pass found 15 more, including new posts starting with no `hot_rank` at all, a TOCTOU double-count race, and a reusable pagination cursor that silently produced wrong pages across different sorts.

Backend Phase 3 — moderation, automod, search, bans

2ddf584, 0c80ff2

Problem solved: no way to report content, no moderator tooling, no automated filtering, no bans, no search.

Required: the moderation module, a real bitmask permission system (replacing Phase 1's placeholder — see Section 3 #10), automod rules attributed to a seeded bot account, a site-admin role, tsvector search, a Caffeine+Redis cache for /hot's first page.

Real bugs: reports/moderation_actions had been declared PARTITION BY RANGE since Phase 1 with zero partitions ever created — this is the first phase to write to either, so every insert would have failed; a follow-up review found 14 more, including cross-community IDORs and an unbounded regex evaluation vulnerable to ReDoS.

Backend Phase 4 — notifications, messaging, media, engagement, account settings

272a035

Problem solved: no notifications, no messaging, no media uploads, no save/hide, no account deletion.

Required: four new modules (notify, a later-replaced message module, engagement, media), the shared OutboxWriter, a dedicated NotificationOutboxWorker.

Real bugs: all 15 code-review findings from the initial build, including missing media ownership/kind validation, no upload size enforcement, and the same partitioned-but-zero-partitions bug, this time on notifications. Also: docker-compose.yml had to swap MinIO for adobe/s3mock after MinIO's Docker Hub image started requiring auth to pull.

Chat replaces classic Private Messages

d59afda

Problem solved: real Reddit has been retiring store-and-forward Private Messages in favor of real-time Chat; Phase 4's message module was deliberately torn out and replaced.

Required: the chat module — STOMP over WebSocket, authenticated at the CONNECT frame (see Section 1).

Real bug: the StompHeaderAccessor.wrap() footgun — see Section 3 #11.

Flair (post flair and user flair)

1de54d3

Problem solved: the `flairs` table existed since Phase 1's very first migration with zero Java code touching it.

Required: mod-managed flair definitions per community, post flair (validated at submission, attached via one batched query — not $N+1$), user flair (self-assignable).

Backend feature-gap backlog, items 1–6 (shipped before the frontend phase)

A dedicated audit — comparing this app against real reddit.com feature-by-feature — found 21 more gaps. Six shipped:

1. Pin/lock a post

8fc958b

`Post.pinned/locked` existed since Phase 1 with no code calling them. Pinned posts deliberately don't fold into `/new//hot`'s own sort order — doing so would require rewriting proven keyset pagination for a cosmetic benefit — so pinned posts get their own small `GET /r/{name}/pinned` list instead.

2. Community rules sidebar

6160c72

`communities.rules` (JSONB) existed since Phase 1, unused. A mod-only whole-list-replace write endpoint — not index-addressed CRUD, since a short rules list is edited as one unit in practice.

3. Per-notification-type mute preferences

f3e340b

Could only mute all notifications or none before. Enforced in `NotificationOutboxWorker` — the single chokepoint every notification type already passes through — rather than in each of the several places that produce a notification.

4. Restricted/private communities

f0f55c2

`communities.type` had allowed these values since Phase 1, but every community was hardcoded public. The single feature that touched the most already-proven call sites in the whole backlog (every post/comment read and write endpoint needed a new access-gate check).

5. Community discovery/browse

4fe5d0b

No way to find a community without already knowing its exact name. The `pg_trgm` trigram index for name search existed since Phase 1, unused.

6. Comment sort options

431f4aa

Comments only ever came back in one fixed order. Verified with real hand-computed Wilson-confidence-score arithmetic — not guessed vote counts (an early attempt with plausible-sounding numbers failed twice before the real math worked).

Frontend phase — F1 through F6 (shipped)

No frontend existed before this phase. Each followed the same plan → build → verify → explicit-go-ahead discipline, verified through a live browser instead of curl scripts.

F1 — Project scaffold + global shell

e02218d

Vite+React+TS SPA, nav shell, in-memory access token with silent refresh, Login/Register. **Found a real gap:** no logout endpoint existed at all — added `POST /api/v1/logout`.

F2 — Home feed

7fcb931

Post cards, vote arrows, sort tabs, infinite scroll. **Found a real gap:** no sitewide feed existed. Built `r/all` as a real pseudo-community (the same convention reddit.com itself uses), with Redis caching.

F3 — Post detail + comments

46f89d1

Nested comment tree, sort dropdown, reply box. **Found the biggest gap of the frontend phase:** there was no way to ever read a comment's replies — once posted, a reply became permanently invisible to every read endpoint. Also fixed a bug where removed root comments were excluded entirely, which would have orphaned their reply subtrees.

F4 — Community page

575bef2

About/rules sidebar, join/leave, pinned posts. **Found a real gap:** `GET /r/{name}/about` had no concept of the viewer, so the frontend couldn't render Join vs. Joined vs. Request Pending.

F5 — Community discovery

6a2c6a6

Browse/search page, working nav-bar search. **Caught an N+1 risk during planning, before writing code** — F4's viewer-context lookup would have meant up to 75 queries per browse page if reused in a loop; batched it first.

F6 — Create post / create community

cb6ff76

The first in-app way to create anything — text/link/image+video posts via presigned upload, flair, NSFW/spoiler, community creation. **Found two real bugs** — the records-and-primitives crash and the CORS header gap — both detailed in Section 3 (#1 and #4), the most instructive bugs of the whole frontend phase.

What's next

F7 — User profile page (overview/posts/comments tabs, karma) is next, followed by F8 (moderation dashboard), F9 (notifications inbox), F10 (chat UI), F11 (settings).

What's intentionally parked — backend features 7 through 20

Paused until the frontend phase (F1–F11) completes: sitewide search; `/api/morechildren` + real comment pagination; rate limiting (`bucket4j` already in `pom.xml`, unused); follow-a-user; gallery posts; avatar upload; link preview/OG-image fetching (flagged as needing SSRF-safe fetching when built); data export (GDPR); polls; trophies; wiki pages; multireddits; community chat rooms; live threads.

Permanently out of scope (an explicit decision, not an oversight)

- **Email verification, password reset, email notifications** — all three need a real email service this project doesn't have; the owner chose to wait for a real Phase 5 rather than build a stub/logged-token workaround.
- **Awards/coins, advertising, developer OAuth platform** — all three conflict with this project's \$0/month, non-commercial premise.

Deferred, not skipped

- **Push notifications** — no mobile app exists in this project to register a device token against.
- **2FA** — deferred by owner decision, to be revisited later.

5. Glossary for Self-Study

Every concept used in this codebase beyond basic CRUD, each with a plain-English explanation and a pointer to real use in this repo. Read the explanation, then go read the cited file.

ArchUnit — a testing library for writing architecture rules as ordinary JUnit tests, statically scanning compiled classes for structural violations ("no cycles between these packages"). A violation fails the build like a broken unit test. `ModuleBoundaryTest.java`; see Section 2.

Modular monolith — a single deployable whose internal code is split into package-per-feature modules with an enforced rule that modules only call each other through service classes, never through another module's repository. See Section 2.

Spring Security filter chain — before a request reaches a controller, it passes through a stack of servlet filters checking things like "is there a valid token," "does CORS allow this." Assembled in `SecurityConfig.filterChain (SecurityConfig.java:52)`, with this project's own filter spliced into a specific position (line 86).

SecurityContext / Authentication — a per-request holder for "who is this request authenticated as." Populated in `JwtAuthFilter.java` via `SecurityContextHolder.getContext().setAuthentication(...)`, read back anywhere via `@AuthenticationPrincipal UUID userId`.

Stateless session (`SessionCreationPolicy.STATELESS`) — unlike a traditional session-cookie app, every request carries its own proof of identity and is re-verified from scratch; nothing is remembered server-side between requests. `SecurityConfig.java:57`.

Argon2id — a deliberately slow, memory-hard password-hashing algorithm, tuned so one real hash takes ~100–200ms but brute-forcing billions of guesses against a stolen hash table becomes prohibitively expensive. `SecurityConfig.java:27–30`.

JWT access token vs. refresh token — two tokens, two jobs: a self-contained signed JWT, fast to verify, short-lived (15 min); and an opaque random value checked against the database on every use, long-lived (30 days), used only to silently obtain a new access token. See Section 2.

httpOnly cookie — a cookie flag telling the browser to never expose a value to page JavaScript, only send it with matching HTTP requests. This project's refresh token lives only here, so even an XSS bug couldn't read it. `AuthController.java`.

CORS preflight, allowedOrigins vs. allowedHeaders — a browser calling a different-origin API first sends an automatic OPTIONS preflight. `allowedOrigins` controls which sites may call the API; `allowedHeaders` separately controls which request headers the real request may carry — a real source of a production bug here (Section 3, #4).

Java records and compact canonical constructors — a record is a concise, immutable data carrier with an auto-generated constructor. Jackson deserializes JSON into a record by calling that constructor directly — an absent field becomes `null`, not a quietly-kept default. A compact constructor (`public Foo { ... }`) intercepts and normalizes values before assignment. See Section 3, #1.

The Outbox pattern — write the triggering fact and a small "this happened" event row in the **same transaction**, so neither can be lost without the other; a background worker later polls and processes events in batches. `OutboxWorker.java`; see Section 2.

@Scheduled and ShedLock — `@Scheduled` runs a method on a timer, independent of any HTTP request. `@SchedulerLock` (`ShedLock`) wraps it in a database-backed lock so only one app instance's invocation actually executes a given tick. `SchedulingConfig.java`.

FOR UPDATE SKIP LOCKED — a SQL clause for safely letting multiple workers grab rows from a queue-like table concurrently: locks the rows a query selects, and tells a concurrent query to simply skip any already-locked row rather than block or double-process it.

Presigned URL (direct-to-storage upload) — the backend never receives a file's raw bytes; it asks the storage SDK to cryptographically sign a time-limited URL authorizing one specific PUT, and the client uploads directly using that URL. See Section 2.

@Transactional — marks a method whose database operations should all succeed or all roll back together, as one atomic unit — the Spring equivalent of SQL's BEGIN/COMMIT/ROLLBACK.

Flyway migrations — numbered, immutable SQL files applied once each, in order, instead of letting Hibernate auto-generate schema changes. `src/main/resources/db/migration/`.

Redis via Spring Data (StringRedisTemplate) — a thin, low-level client for raw string values in Redis, used here by hand rather than via the `@Cacheable` annotation abstraction, so fail-open behavior and TTL layering can be controlled explicitly. `FeedCacheService.java`.

STOMP over WebSocket — `WebSocket` upgrades an HTTP connection into a persistent, two-way connection; STOMP is a lightweight messaging sub-protocol on top, giving `@MessageMapping` methods a role analogous to `@RequestMapping` for REST. `ChatWebSocketConfig.java`.

Wilson score (confidence-interval ranking) — a statistical formula for ranking by "upvote ratio" that accounts for sample size, so 2 upvotes/0 downvotes doesn't automatically outrank 50 up/49 down. `RankFormulas.java`.

UUIDv7 — a time-ordered UUID variant: sorts naturally by creation time and clusters better in a database index than a fully random UUIDv4, while staying globally unique. `UuidV7Generator.java`.

Keyset pagination (vs. OFFSET) — instead of `LIMIT 25 OFFSET 500` (slower as the offset grows, can skip/duplicate rows if data changes between pages), a cursor carries the last-seen row's own sort key(s):
`WHERE (created_at, id) < (cursor) ORDER BY ... LIMIT 25.` `common/paging/Cursor.java`.

The one habit worth taking away from this whole codebase

Across almost every phase of this project's history, the single most common type of real gap wasn't a logic bug — it was **a database column or table that had existed since the very first migration, with zero application code ever using it** (flair, pin/lock, community rules, the trigram search index, per-type notification preferences all started this way). The schema was designed up front for the whole project, well ahead of the code that would eventually use each piece of it. When asked to add a feature here, checking `src/main/resources/db/migration/` first for something that already exists but is unused is a genuinely useful habit — more often than not, the column is already there and the feature is "ship code," not "design schema."